

Blueprint Project Guide

5 production-grade developer tool projects — each with tech stack, architecture decisions, build workflow, hard technical problems, and ready-to-paste resume bullets.

5

Projects

30+

Weeks of Build Time

5

npm/pip Packages

20+

Tech Stack Items

Prepared for · Rohan Vishwakarma · Full Stack Developer · 2025

TABLE OF CONTENTS

- 01** **Downtime.so** — Incident Communication Infrastructure
 - 02** **Pipedream-lite** — Visual Webhook & Automation Pipeline Builder
 - 03** **Schemalab** — Live Database Schema Collaboration Tool
 - 04** **Logsmith** — Structured Logging UI & Alerting
 - 05** **Vaultkey** — Secrets & Environment Variable Manager
-

Downtime.so

Incident Communication Infrastructure

Affordable, developer-first status pages & incident timelines for indie hackers and small teams.

5–6 weeks	■■■■■	Paid Tool Territory
Est. Build Time	Difficulty	Market Position

■ THE ACUTE PAIN

Your app crashes at 2am. Users flood support. You have no status page, no incident timeline, no automated alerts. Statuspage.io costs \$299/month — way out of reach for indie devs. Instatus exists but feels like a form, not a product.

■ WHAT IT ACTUALLY IS

A hosted status page platform — get status.yourapp.com in minutes, post real-time incident updates, automatically notify subscribers via email/SMS, and update your status programmatically from your own monitoring scripts via a clean REST API.

■ UNIQUE FEATURES THAT STAND OUT

- Incident **story format** — live narrative timeline users follow like a thread, drastically reducing support tickets during outages
- One-line React embed — drop `<DowntimeBanner />` into your app, it auto-shows a banner during active incidents
- **API-first** — update status from a cron job, GitHub Action, or Datadog webhook without touching the UI
- Multi-service architecture — one account, multiple services, each with independent status histories
- Fan-out **notification pipeline** — email + SMS blast to N subscribers the moment an incident is created

■■ FULL TECH STACK

Next.js 14	TypeScript	Node.js	PostgreSQL	Prisma ORM
Redis	BullMQ	Resend API	Twilio SMS	Tailwind CSS
Framer Motion	Vercel	Railway	REST API	JWT Auth

■ HARD TECHNICAL PROBLEMS

Webhook Ingestion	Receive pings from external monitors (UptimeRobot, Datadog) and translate them into status changes reliably
-------------------	---

Fan-out Notifications	Efficiently email/SMS all subscribers on incident create without blocking the main thread — use BullMQ job queues
Status Page Uptime	Public status page must stay live even when the main app is degraded — deploy as a separate static/edge service
Rate Limiting	Prevent abuse of the public API (status updates) using Redis sliding window rate limiting per API key

■■ STEP-BY-STEP BUILD WORKFLOW

01

Project Setup & Auth

Scaffold Next.js 14 with TypeScript. Set up PostgreSQL + Prisma schema: User, Organization, Service, Incident, IncidentUpdate, Subscriber. Implement JWT auth with refresh tokens.

02

Status Page Engine

Build the public status.domain.com page as a separate Next.js route group. Fetch service statuses server-side. Implement real-time updates via Server-Sent Events so the page auto-refreshes.

03

Incident Management UI

Dashboard to create/update incidents. Each incident has a status (investigating → identified → monitoring → resolved) and an update timeline. Store timestamps for each transition.

04

Notification Pipeline

On incident create/update, push a job to BullMQ. Worker picks it up, fetches all subscribers, sends Resend emails + Twilio SMS in batches. Log delivery status per subscriber.

05

REST API + Webhooks

Build /api/v1/services, /api/v1/incidents endpoints. Accept incoming webhook pings from monitors. Map monitor status to service status automatically.

06

React Embed Component

Publish @downtime/react as an npm package. Component polls your status API every 60s and conditionally renders an incident banner with latest update message.

07

Polish & Deploy

Add Framer Motion animations to the status page timeline. Deploy main app to Vercel, worker service to Railway. Set up custom domain for status pages.

■ RESUME BULLETS (COPY-PASTE READY)

- Built a multi-tenant incident communication platform serving real-time status updates via SSE to N concurrent subscribers
- Architected a BullMQ fan-out notification pipeline delivering email + SMS alerts to subscribers on incident creation
- Published an open-source React embed component () allowing one-line status integration for any web app
- Implemented webhook ingestion system accepting pings from UptimeRobot/Datadog and auto-updating service health status

Pipedream-lite

Visual Webhook & Automation Pipeline Builder

Wire APIs together visually. No Zapier subscription. No self-hosted VM. Just draw the flow and ship it.

7–8 weeks	■■■■■	Paid Tool Territory
Est. Build Time	Difficulty	Market Position

■ THE ACUTE PAIN

You want: when a GitHub issue is created → post to Slack + log to Notion + send email. Zapier charges \$20/month and throttles runs. n8n requires a whole self-hosted VM. There's no fast, affordable middle ground for developers who just want to wire APIs without writing a full backend.

■ WHAT IT ACTUALLY IS

A visual drag-and-drop pipeline builder where each node is an HTTP trigger, transformer, condition, or action. Build a flow in the browser, deploy it, and it runs as a persistent webhook listener. Full execution logs. Code escape hatch per node. Version history.

■ UNIQUE FEATURES THAT STAND OUT

- **Code escape hatch** — every node has a 'write custom JS' mode; power users aren't boxed in by GUI limits
- **Pipeline versioning** — every deploy is versioned, roll back to v3 with one click
- **Execution logs** with full request/response per run — debugging is the #1 frustration with existing tools
- Export as **standalone Node.js script** — you own your logic, not locked into the platform
- **React Flow** graph editor — visually stunning, gets posted on Twitter

■■ FULL TECH STACK

Next.js 14	TypeScript	React Flow	Node.js	PostgreSQL
Prisma ORM	Redis	BullMQ	Docker	VM2 (sandbox)
Tailwind CSS	Monaco Editor	Framer Motion	WebSockets	REST API

■ HARD TECHNICAL PROBLEMS

Persistent Listeners	Long-running webhook listeners — not serverless. Use Node.js HTTP server per pipeline deployment, managed via pm2 or a worker pool.
----------------------	---

Graph Cycle Detection	Before saving a pipeline, run DFS on the node graph to detect cycles. An infinite loop pipeline would crash the worker.
Sandboxed JS Execution	Run user's custom node code in vm2 sandbox with a 5s timeout and no fs/network access beyond whitelisted fetch calls.
Execution Log Storage	Each pipeline run logs every node's input/output payload. Store in PostgreSQL with JSONB columns. Add TTL-based cleanup after 30 days.

■■ STEP-BY-STEP BUILD WORKFLOW

01

Graph Editor

Integrate React Flow. Define node types: Trigger (webhook), HTTP Request, Transform (JS), Filter/Condition, Action (Slack/email/webhook). Each node has a config panel sidebar.

02

Pipeline Schema

Design DB schema: Pipeline, PipelineVersion, Node, Edge, PipelineRun, NodeExecution. A version is a snapshot of nodes+edges at deploy time.

03

Worker Runtime

Build a Node.js worker that loads a pipeline version, sets up an HTTP endpoint for the trigger node, and executes nodes in topological order on each incoming request.

04

Sandboxed Code Nodes

For Transform nodes with custom JS, use vm2 to execute user code with a 5s timeout. Pass node's input as a variable. Return value becomes node output.

05

Execution Logging

After each node runs, write input/output/status/duration to NodeExecution table. Stream live execution status to the UI via WebSocket during a run.

06

Version Control

On 'Deploy', create a new PipelineVersion with a snapshot of current nodes/edges. Previous version keeps running until new one is healthy.

07

UI Polish

Add animated edges in React Flow showing data flow direction. Execution log viewer with collapsible JSON payloads. One-click 'Export as Node.js script' button.

■ RESUME BULLETS (COPY-PASTE READY)

- Built a visual automation pipeline builder using React Flow with persistent webhook listeners managing concurrent pipeline executions
- Implemented sandboxed JavaScript execution per pipeline node using vm2 with 5s timeout and isolated network access
- Designed a pipeline versioning system with rollback capability — each deploy snapshots the full node/edge graph
- Streamed real-time execution logs per node to the UI via WebSocket, with full request/response payload storage in JSONB

Schemalab

Live Database Schema Collaboration & Docs Tool

Connect your PostgreSQL database. Get a live, beautiful, always-accurate ER diagram your team can annotate.

6–7 weeks	■■■■■	Paid Tool Territory
Est. Build Time	Difficulty	Market Position

■ THE ACUTE PAIN

Your team's database schema lives in a migration file nobody reads, a Notion doc that's 3 months stale, and someone's head. New engineers take weeks to understand the data model. DBDiagram is static — paste SQL, get a PNG. There's no live, collaborative, always-accurate schema visualization.

■ WHAT IT ACTUALLY IS

Connect your PostgreSQL DB (read-only credentials). Schemalab auto-generates a beautiful, interactive ER diagram — live and always accurate. Teams annotate directly on the diagram. Schema diff between environments. Change impact analysis. Shareable public links.

■ UNIQUE FEATURES THAT STAND OUT

- **Schema diff view** — compare staging vs production schema side-by-side, see missing columns/indexes instantly
- **Change impact analysis** — click a table, see every table with a foreign key dependency on it highlighted
- **Inline docs** on the diagram itself — not in Notion, not in a README, right on the column
- Share a **public read-only schema URL** with contractors/new hires without giving DB access
- Collaborative annotations via **Socket.io** — like Figma comments, on your database diagram

■■ FULL TECH STACK

Next.js 14	TypeScript	React Flow	Node.js	PostgreSQL
Prisma ORM	Socket.io	information_schema	pg_catalog	Tailwind CSS
Framer Motion	Dagre (layout)	JWT Auth	Zod	

■ HARD TECHNICAL PROBLEMS

Schema Introspection	Query information_schema.columns, table_constraints, key_column_usage, referential_constraints to reconstruct full schema with FK relationships.
----------------------	--

Large Diagram Layout	100+ table schemas become unreadable. Use Dagre auto-layout algorithm to compute node positions. Allow user to drag and save custom positions.
Real-time Collab	Multiple team members annotating simultaneously — use Socket.io rooms per schema. Broadcast annotation create/update/delete events. Persist to DB.
Live Refresh	Detect schema changes since last sync using a checksum of the introspected schema. Alert users when schema has drifted from the diagram.

■■ STEP-BY-STEP BUILD WORKFLOW

01

DB Connection & Introspection

Accept read-only PostgreSQL connection string. Use pg client to query information_schema and pg_catalog. Extract tables, columns, types, constraints, foreign keys, indexes.

02

Schema Data Model

Store introspected schema in your own DB: Schema, SchemaTable, SchemaColumn, SchemaRelationship. Each sync creates a new SchemaSnapshot for diff comparison.

03

React Flow Diagram

Render tables as custom React Flow nodes. Columns listed inside node card. FK relationships rendered as animated edges. Use Dagre for initial auto-layout.

04

Schema Diff Engine

Compare two SchemaSnapshots. Produce a diff: added tables, dropped tables, added/removed columns, type changes. Render diff as a color-coded overlay on the diagram.

05

Collaborative Annotations

Each column/table can have Annotation objects (text notes + author). Socket.io broadcasts annotation events to all connected clients in the same workspace room.

06

Public Share Links

Generate a signed public URL for a schema snapshot. Render a read-only version of the diagram without the workspace sidebar. Password-protect optionally.

07

Polish

Add smooth Framer Motion transitions when new tables appear on sync. Minimap for large schemas. Search-to-highlight a table across the diagram.

■ RESUME BULLETS (COPY-PASTE READY)

- Built a live ER diagram tool that introspects PostgreSQL schemas via `information_schema` and `pg_catalog`, rendering 100+ table schemas with Dagre auto-layout
- Implemented schema diff engine comparing two snapshots to surface column additions, type changes, and dropped tables with visual color-coded overlays
- Added real-time collaborative annotations using Socket.io workspace rooms, enabling multiple engineers to document the schema simultaneously
- Engineered change impact analysis that highlights all FK-dependent tables when a table is selected, reducing risk of breaking schema changes

Logsmith

Structured Logging UI & Alerting for Self-Hosted Apps

npm install logsmith. Ship logs from your app. Get a beautiful real-time explorer. No Datadog bill.

6–7 weeks	■■■■■	Paid Tool Territory
Est. Build Time	Difficulty	Market Position

■ THE ACUTE PAIN

You have a Node.js app on a VPS. Something breaks. You SSH in, tail -f logs/app.log, squint at JSON blobs, and grep blindly. Datadog costs \$15/host/month minimum. There's no affordable, self-contained logging UI for indie devs and small teams.

■ WHAT IT ACTUALLY IS

A lightweight logging SDK + hosted UI. One npm install, one line of code, and your app starts sending structured logs to a beautiful real-time explorer. Filter by level, service, time range, and custom tags. Set alert rules that ping Slack or email when error rate spikes.

■ UNIQUE FEATURES THAT STAND OUT

- SDK is **framework-agnostic** — works with Express, Fastify, Next.js API routes, anything
- **Session replay for logs** — trace a single user session end-to-end across all log lines via auto-injected session ID
- **Anomaly alerting** — not just 'error count > 10' but 'error rate 3x higher than same time yesterday'
- **30-day retention** then auto-archive to S3/R2 — you control your data, no vendor lock-in
- **TimescaleDB** hypertables for millisecond log queries even at millions of rows

■■ FULL TECH STACK

Next.js 14	TypeScript	Node.js	PostgreSQL	TimescaleDB
Prisma ORM	Redis	BullMQ	Resend API	Tailwind CSS
Recharts	Server-Sent Events	npm SDK package	S3/R2	Zod

■ HARD TECHNICAL PROBLEMS

High-throughput Ingestion	Logs arrive in bursts (deploy event = 1000 logs/sec). Buffer incoming logs in Redis, flush to TimescaleDB in batches every 500ms to avoid write amplification.
---------------------------	--

TimescaleDB Queries	Time-series queries on millions of rows need hypertables + chunk exclusion. Create indexes on (time, level, service) for the most common filter combinations.
Real-time Log Streaming	The log explorer streams new logs as they arrive. Use Server-Sent Events — much cheaper than WebSockets for unidirectional streaming, no connection overhead.
Anomaly Detection	Compare current 15-min window error rate vs same window yesterday. Store aggregated buckets (per 15 min) in a materialized TimescaleDB continuous aggregate view.

■■ STEP-BY-STEP BUILD WORKFLOW

01

SDK Development

Build the logsmith npm package. Export a Logger class with .info(), .warn(), .error(), .debug() methods. Auto-inject session ID from AsyncLocalStorage context. Batch logs, send to ingestion API every 2s.

02

Ingestion Pipeline

POST /api/ingest endpoint accepts log batches. Validate schema with Zod. Write to Redis buffer list. BullMQ worker flushes buffer to TimescaleDB hypertable every 500ms.

03

TimescaleDB Setup

Create logs hypertable partitioned by time (1 day chunks). Create indexes: (time DESC, level), (time DESC, service), (session_id). Set up continuous aggregates for 15-min error rate buckets.

04

Log Explorer UI

Build filterable log table: level selector, service multi-select, time range picker, free-text search. Render log lines with syntax-highlighted JSON. Add real-time append via SSE.

05

Session Trace View

Click a session_id in any log line → open a drawer showing all log lines for that session across all services in chronological order. The 'session replay for logs' feature.

06

Alerting Engine

Alert rules stored in DB: condition type (threshold / anomaly), channel (Slack webhook / email), cooldown period. BullMQ runs evaluation job every 5 minutes, sends alerts via Resend.

07

Archive & Retention

Scheduled job runs nightly. Exports logs older than 30 days to S3/R2 as gzipped JSONL files, then deletes from TimescaleDB. Retention config per workspace.

■ RESUME BULLETS (COPY-PASTE READY)

- Built and published a framework-agnostic Node.js logging SDK that batches and ships structured logs with auto-injected session context
- Architected a high-throughput ingestion pipeline using Redis buffering + BullMQ batch writes to TimescaleDB, handling burst traffic without write pressure
- Implemented real-time log streaming via Server-Sent Events and an anomaly alerting engine comparing error rates against historical baselines
- Designed session trace view enabling end-to-end user session reconstruction across microservices from a single session ID

Vaultkey

Secrets & Environment Variable Manager for Dev Teams

Git for your .env files. AES-256 encrypted. CLI-first. Full audit log of who saw what and when.

5–6 weeks	■■■■■	Paid Tool Territory
Est. Build Time	Difficulty	Market Position

■ THE ACUTE PAIN

.env files get committed to GitHub. Secrets get shared over Slack DMs. When someone leaves the team, you have no idea which secrets they accessed. Doppler solves this but feels enterprise-heavy and costs money. Most small teams just suffer and hope nothing gets leaked.

■ WHAT IT ACTUALLY IS

A secrets manager with a CLI + web UI. Store environment variables per project and environment (dev/staging/prod). Pull them with 'vaultkey pull --env production'. Full audit log of every access. Secret rotation reminders. GitHub Actions integration. AES-256 encryption at rest.

■ UNIQUE FEATURES THAT STAND OUT

- CLI is the primary interface — vaultkey push / pull / diff feel like git for your secrets
- Secret rotation reminders — flag secrets older than 90 days as stale, one-click re-generation for JWT secrets / API keys
- Access log per secret — 'this key was accessed by Rohan on Nov 3 from IP 103.x.x.x'
- GitHub Actions integration — inject secrets into CI pipelines without storing them in repo Settings
- Append-only audit log — tamper-evident, cryptographically chained log of all access events

■ FULL TECH STACK

Next.js 14	TypeScript	Node.js	PostgreSQL	Prisma ORM
Commander.js (CLI)	AES-256-GCM	Node crypto module	JWT	Tailwind CSS
Framer Motion	Zod	Resend API	GitHub OAuth	npm CLI package

■ HARD TECHNICAL PROBLEMS

Encryption at Rest	Encrypt each secret value with AES-256-GCM. Derive a per-workspace encryption key from a master key using PBKDF2. Never store plaintext or the raw master key in DB.
CLI Auth Flow	Device token flow like GitHub CLI: CLI opens browser to /auth/device, user approves, CLI polls for token. Token stored in OS keychain (~/.vaultkey/credentials).
Append-only Audit Log	AuditLog table with INSERT-only policy (no UPDATE, no DELETE at DB level). Each row includes a hash of the previous row — tamper-evident chain.
GitHub Actions Integration	A custom GitHub Action that calls the Vaultkey API with a machine token, fetches secrets for the environment, and exports them as masked env vars for the workflow.

■■ STEP-BY-STEP BUILD WORKFLOW

01

Encryption Layer

Implement AES-256-GCM encryption/decryption utilities. Generate a data encryption key (DEK) per workspace, wrapped by a master key. Test with known plaintext/ciphertext vectors before storing any real secrets.

02

Core API

Build REST API: POST /secrets (encrypt + store), GET /secrets (decrypt + return), DELETE /secrets, GET /audit-log. All endpoints require valid JWT. Rate limit GET /secrets to prevent bulk extraction.

03

Audit Logging Middleware

Every API call that accesses a secret writes an AuditEntry: user_id, secret_id, action, ip_address, user_agent, timestamp, prev_entry_hash. Wire this as Express middleware.

04

CLI Development

Build vaultkey CLI with Commander.js. Commands: login (device flow), push (read local .env, upload), pull (fetch + write .env), diff (compare local vs remote), ls (list secrets). Publish to npm.

05

Web Dashboard

Secrets explorer: list secrets (show key names, hide values behind blur), reveal with re-auth, edit, delete. Audit log table with filters. Environment switcher (dev/staging/prod) per project.

06

Rotation & Alerts

Scheduled job checks secret age daily. Secrets older than 90 days get flagged as STALE. Send email reminders to workspace admins. Generate new JWT secrets / random API keys on one-click rotation.

07

GitHub Actions

Publish vaultkey/inject-secrets GitHub Action to GitHub Marketplace. Action authenticates with a machine token, calls GET /secrets, exports each as `${env.SECRET_NAME}` masked in logs.

■ RESUME BULLETS (COPY-PASTE READY)

- Built a secrets manager with AES-256-GCM encryption at rest, per-workspace key derivation via PBKDF2, and a CLI auth device flow mirroring GitHub CLI
- Implemented a tamper-evident append-only audit log using cryptographic hash chaining of sequential audit entries, with INSERT-only DB policy
- Published vaultkey CLI to npm with push/pull/diff commands, enabling .env management workflows identical to git version control
- Authored a custom GitHub Action to inject Vaultkey secrets as masked environment variables into CI/CD pipelines without storing credentials in repo settings

BUILD PRIORITY MATRIX

Project	Tech Depth	Visual Wow	Originality	Startup Signal	Timeline
Downtime.so	■■■■■	■■■■■	■■■	■■	5–6 wks
Pipedream-lite	■■■■■	■■■■■	■■■■■	■■■	7–8 wks
Schemalab	■■■■■	■■■■■	■■■■■	■■■	6–7 wks
Logsmith	■■■■■	■■■■■	■■■■■	■■■	6–7 wks
Vaultkey	■■■■■	■■■	■■■■■	■■■	5–6 wks

■ Recommended Build Order

Start with Vaultkey (fastest, teaches encryption & CLI) → then Logsmith (SDK + ingestion pipeline) → then Schemalab or Pipedream-lite (highest visual impact for interviews)

Built with intention. Shipped with precision.